

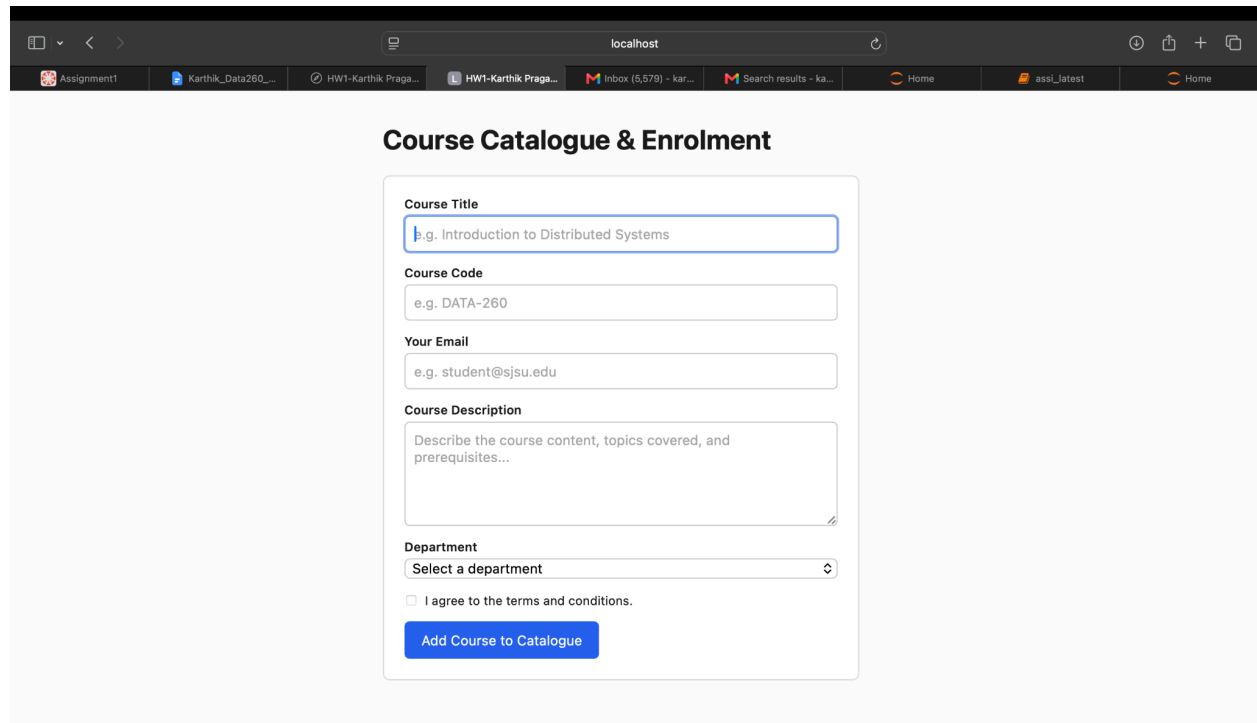
DATA - 260
HomeWork-1
Name: Karthik Pragada
SJSU ID: 018768360

Value	Definition
SID	8360
PORT_BASE	8260
PREFIX	s8360
SEED	8360
VERIFY_SEED	268360
DOMAIN_ID	0
HARDWARE	MacBook Air, Apple M4, 16 GB RAM
Local Model Used	qwen3:8b Ollama Model
Tagged commit hash	c0da78fd8a89d6613221908c9faef85f9d380a5e
Git repo	https://github.com/prags8585/data260-8360/tree/main

Part 1 (HTML, JAVASCRIPT & AWS)

1 - HTML

Created a HTML Web Page as per the instructions from the assignment pdf.
For my Domain_ID = 0 (Campus course catalogue and enrolment form)



The screenshot shows a web browser window with the address bar set to 'localhost'. The browser's tab bar contains several tabs, including 'Assignment1', 'Karthik_Data260...', 'HW1-Karthik Praga...', and 'HW1-Karthik Praga...'. The main content area displays a form titled 'Course Catalogue & Enrolment'. The form is a light gray box with a white background. It contains the following fields: 'Course Title' with a placeholder 'e.g. Introduction to Distributed Systems', 'Course Code' with a placeholder 'e.g. DATA-260', 'Your Email' with a placeholder 'e.g. student@sjsu.edu', 'Course Description' with a placeholder 'Describe the course content, topics covered, and prerequisites...', and a 'Department' dropdown menu with the text 'Select a department'. Below these fields is a checkbox labeled 'I agree to the terms and conditions.' and a blue button labeled 'Add Course to Catalogue'.

Local host image



The form consists of -

Course title : Name of the Course

Course Code : Subject Code

Email: Student email

Course Description: Content should have >25 characters

Department: a Drop down menu with (Computer Science, DataScience, Business, engineering)

The Course Title input auto-focuses on page load, and the Department dropdown starts on a disabled empty placeholder so the required attribute is meaningful

“Add Course to Catalogue” is the submit button which helps to submit the form.

Below is an example of a submission

Assignment1

Karthik_Data260...

HW1-Karthik Praga...

HW1-Karthik Praga...

Inbox (5,579) - kar...

Search results - ka...

Home

assi_latest

Home

localhost

Course Catalogue & Enrolment

Course Title

Course Code

Your Email

Course Description

Describe the course content, topics covered, and prerequisites...

Department

Select a department

☐ I agree to the terms and conditions.

Add Course to Catalogue

Submitted Course

Course Title	Introduction to Distributed system
Course Code	DATA- 260
Email	karthik.pragada@sjsu.edu
Description	Good Course, Industry Oriented, Very Helpful.
Department	Data Science
Agreed to Terms	true
Submission Date	Wed Aug 26 2026 23:28:15 GMT-0700 (Pacific Daylight Time)
Submission #	1

Below is the code screenshot of **Index.html**

```

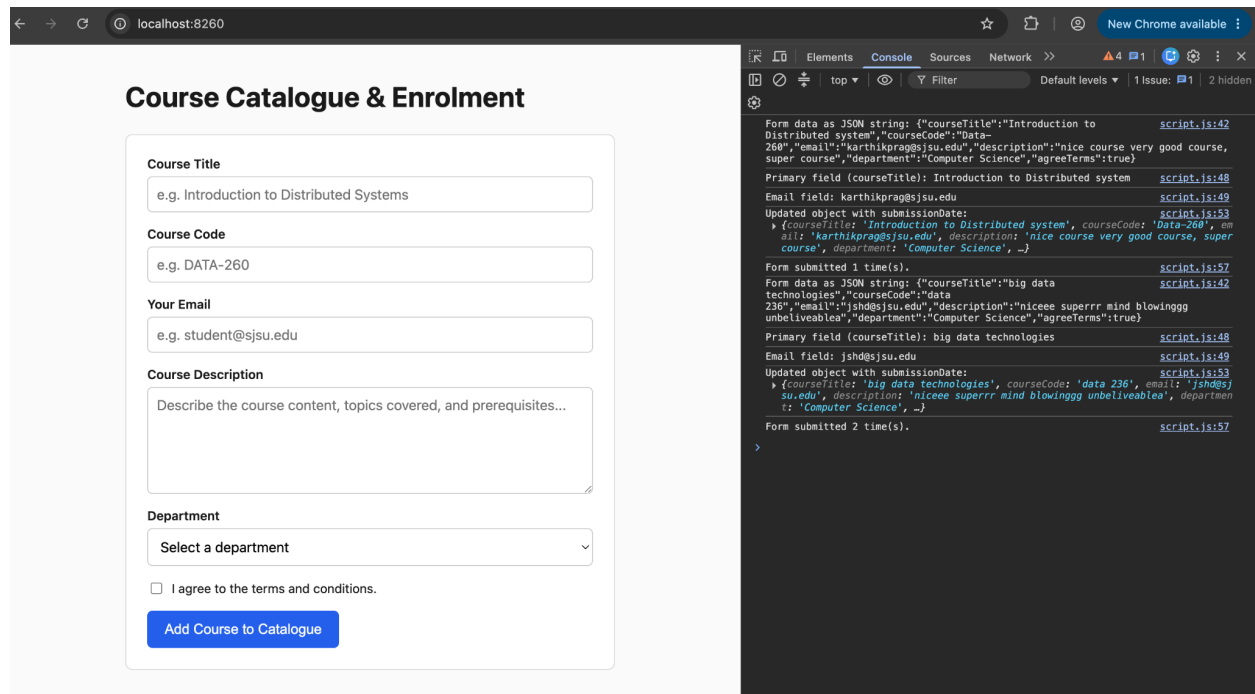
# style.css
index.html
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4 <meta charset="UTF-8">
5 <meta name="viewport" content="width=device-width, initial-scale=1.0">
6 <title>HW1-Karthik Pragada</title>
7 <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10 <h1>Course Catalogue & Enrolment</h1>
11
12 <form id="courseForm">
13 <div class="field">
14 <label for="courseTitle">Course Title</label>
15 <input
16   type="text"
17   id="courseTitle"
18   name="courseTitle"
19   placeholder="e.g. Introduction to Distributed Systems"
20   required
21   autofocus
22 >
23 </div>
24
25 <div class="field">
26 <label for="courseCode">Course Code</label>
27 <input
28   type="text"
29   id="courseCode"
30   name="courseCode"
31   placeholder="e.g. DATA-260"
32   required
33 >
34 </div>
35
36 <div class="field">
37 <label for="email">Your Email</label>
38 <input
39   type="email"
40   id="email"
41   name="email"
42   placeholder="e.g. student@sjsu.edu"
43   required
44 >
45 </div>
46
47 <div class="field">
48 <label for="description">Course Description</label>
49 <textarea
50   id="description"
51   name="description"
52   placeholder="Describe the course content, topics covered, and prerequisites..."
53   required

```

Style.css

```
# style.css  x  < index.html
# style.css > body
1  body {
2    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, sans-serif;
3    max-width: 560px;
4    margin: 40px auto;
5    padding: 0 20px;
6    color: #1a1a1a;
7    background: #fafafa;
8  }
9
10  h1 {
11    font-size: 2rem;
12    margin-bottom: 24px;
13  }
14
15  form {
16    background: #fff;
17    border: 1px solid #ddd;
18    border-radius: 8px;
19    padding: 24px;
20  }
21
22  .field {
23    margin-bottom: 16px;
24    display: flex;
25    flex-direction: column;
26  }
27
28  .field label {
29    font-weight: 600;
30    margin-bottom: 6px;
31    font-size: 0.9rem;
32  }
33
34  input[type="text"],
35  input[type="email"],
36  textarea,
37  select {
38    padding: 10px;
39    border: 1px solid #ccc;
40    border-radius: 6px;
41    font-size: 1rem;
42    font-family: inherit;
43  }
44
45  textarea {
46    min-height: 100px;
47    resize: vertical;
48  }
49
50  .checkbox-field {
```

2 - JavaScript



When I click "Add Course to Catalogue," the browser reloads the page and sends the data to a server which is immediately blocked by `event.preventDefault()` at the top of the submit handler in `script.js`, because this is a client-side-only demo with no backend to receive it.

The script then reads the current value of each of the six form fields (title, code, email, description, department, and the checkbox's checked state) directly off the DOM elements. Those values are handed to `validateForm`, an arrow function that runs two checks in order: first whether the description is longer than 25 characters, and second whether the checkbox is checked. The moment either check fails, an `alert()` pops up explaining what's wrong and the function returns `false`, which stops everything else from running (no logging, no table, no reset).

If both checks pass, the six values are packed into a plain JS object, and that object is immediately serialized with `JSON.stringify()` into a string, which gets logged to the console, this is meant to simulate what I actually send over the network to a real API.

That string is then parsed back with `JSON.parse()` into a fresh object (`parsedObject`), from which `courseTitle` and `email` are pulled out by destructuring and logged individually. Next, the spread operator copies every key of `parsedObject` into a new object while adding one more key, `submissionDate`, set to the current timestamp, this updated object is logged too. A closure-based counter (`trackSubmission`) works like a function that remembers a private count variable across calls which is then invoked to increment and return how many times the form has been successfully submitted, and that count is logged as well.

Finally, the same updated object plus the submission count get rendered as rows in the results table below the form (the piece we just added for visual confirmation), the form is reset back to

its empty/default state, and focus is returned to the Course Title field so it's ready for another entry which mirrors the required autofocus behavior on page load.

Below is the Code snippet of [Script.js](#)

```
# style.css  < index.html  JS script.js  X
JS script.js > FIELD_LABELS
1  const courseForm = document.getElementById('courseForm');
2
3  // arrow function validation
4  const validateForm = (description, agreeTerms) => {
5    if (description.trim().length <= 25) {
6      alert('Course description must be more than 25 characters long. ');
7      return false;
8    }
9    if (!agreeTerms) {
10     alert('Please agree to the terms and conditions before submitting. ');
11     return false;
12   }
13   return true;
14 };
15
16 // closure tracking submission count
17 const createSubmissionCounter = () => {
18   let count = 0;
19   return () => {
20     count += 1;
21     return count;
22   };
23 };
24 const trackSubmission = createSubmissionCounter();
25
26 const FIELD_LABELS = {
27   courseTitle: 'Course Title',
28   courseCode: 'Course Code',
29   email: 'Email',
30   description: 'Description',
31   department: 'Department',
32   agreeTerms: 'Agreed to Terms',
33   submissionDate: 'Submission Date',
34 };
35
36 const renderResultTable = (updatedObject, submissionCount) => {
37   const tbody = document.querySelector('#resultTable tbody');
38   tbody.innerHTML = '';
39
40   const rows = { ...updatedObject, submissionCount };
41   Object.entries(rows).forEach(([key, value]) => {
42     const tr = document.createElement('tr');
43     const th = document.createElement('th');
44     th.textContent = key === 'submissionCount' ? 'Submission #' : (FIELD_LABELS[key] || key);
45     const td = document.createElement('td');
46     td.textContent = String(value);
47     tr.append(th, td);
48     tbody.append(tr);
49   });
50 }
```

Docker

Docker is used to package the Course Catalogue web app (index.html, style.css, script.js) into a portable container so it runs identically anywhere on the laptop or later on AWS. A Dockerfile starts from a minimal python:3.12-slim base image, copies in the three app files, and runs Python's built-in http.server on port 8260 (my assigned PORT_BASE). Running `docker build -t data260-8360-hw1 .` packages this into a single image, and `docker run -d -p 8260:8260 --name hw1-app data260-8360-hw1` starts it as a container, mapping port 8260 inside the container to port 8260 on the machine so `http://localhost:8260` serves the app. This same image is then reused unchanged for the AWS ECS deployment and is pushed to Amazon ECR and run as a Fargate task so, the container behaves identically whether it's running locally or in the cloud, which is the whole point of containerizing it in the first place.

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ
0.03% / 1000% (10 CPUs available)

Container memory usage ⓘ
16.23MB / 11.4GB

[Show charts](#)

Only show running containers

Delete

	Name	Container ID	Image	Port(s)	CPU (%)	Last st	Actions
☐	sql-client-run-17	9863530902f0	learn-apact		0%	5 mont	
☐	hadoop343	31fa75e50ea0	apache/har		0%	6 mont	
☐	sandbox-proxy	b27fc646fe0f	hortonwork	4200:4200 Show all ports (3)	0%	6 mont	
☐	sandbox-hdp	49586bfa0b6a	hortonwork	2222:22 Show all ports (5)	0%	6 mont	
☒	hw1-app	898e0fd9dfce	data260-83	8260:8260	0.03%	10 hou	
☐	aki	-	-	-	0%	4 mont	
☐	bigdata_project	-	-	-	0%	4 mont	
☐	learn-apache-flir	-	-	-	0%	5 mont	

Selected 1 of 16

Containers

Images

Volumes

Kubernetes

Builds

Models

MCP Toolkit BETA

Docker Hub

Docker Scout

Extensions

Images [Give feedback](#)

LocalMy Hub

41.04 GB / 72.89 GB in use 28 images

Last refresh: 1 day ago

Delete

Space to be reclaimed 159 MB

	Name	Tag	Image ID	Created	Size	Actions
☐	bde2020/hadoop-resour	2.0.0-hadoop3.2.1-java8	91ca2afe8616	7 years ago	2.05 GB	
☐	bde2020/hadoop-namei	2.0.0-hadoop3.2.1-java8	51ad9293ec52	7 years ago	2.05 GB	
☐	bde2020/hadoop-histor	2.0.0-hadoop3.2.1-java8	ab2971d5f8c3	7 years ago	2.05 GB	
☐	bde2020/hadoop-datan	2.0.0-hadoop3.2.1-java8	ddf6e9ad55af	7 years ago	2.05 GB	
☐	hortonworks/sandbox-h	3.0.1	7b767af7b420	8 years ago	49.08 GB	
☐	hortonworks/sandbox-p	1.0	42e4cfbcbb76	8 years ago	163.64 MB	
☐	6413794992	Copy to clipboard	e077e6b57674	10 hours ago	202.5 MB	
☒	data260-8360-hw1	latest	e077e6b57674	10 hours ago	202.5 MB	

Selected 1 of 30

```
HW1_DATA260

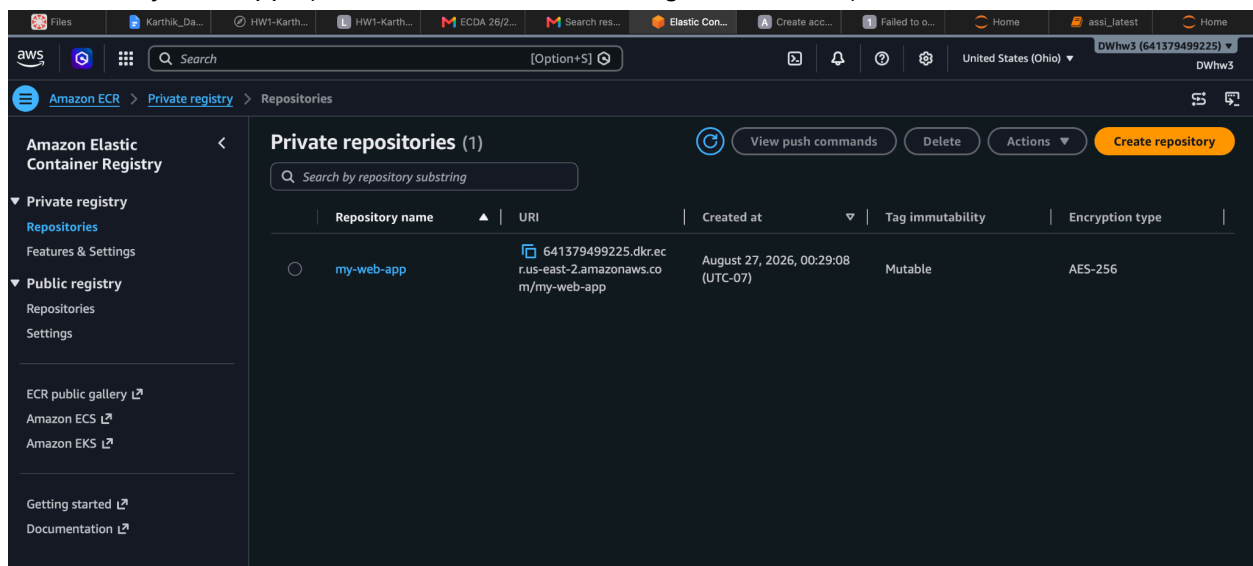
# style.css  <> index.html  JS script.js  Dockerfile x

Dockerfile
1 FROM python:3.12-slim
2
3 WORKDIR /app
4 COPY index.html style.css script.js ./
5
6 EXPOSE 8260
7
8 CMD ["python", "-m", "http.server", "8260"]
9
```

3- AWS ECR DEPLOYMENT

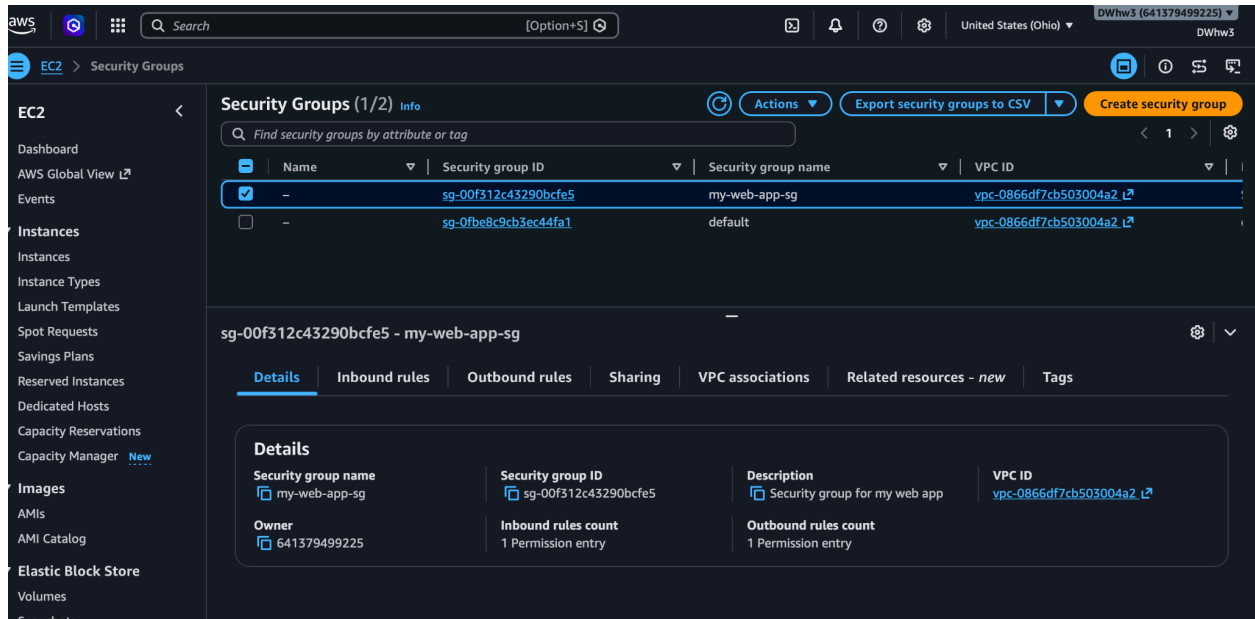
ECR Elastic Container Registry

Created “my-web-app” (account 641379499225, region us-east-2).



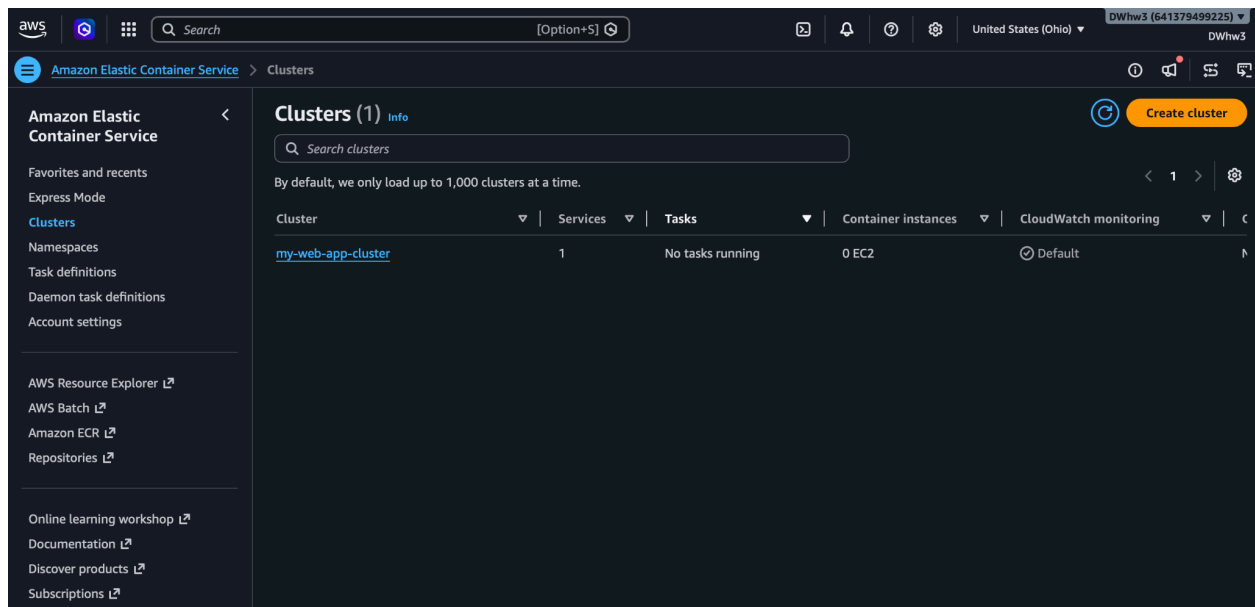
Security groups

Created my-web-app-sg with inbound Custom TCP port 8260 open from 0.0.0.0/0 since the container listens on 8260.



ECS

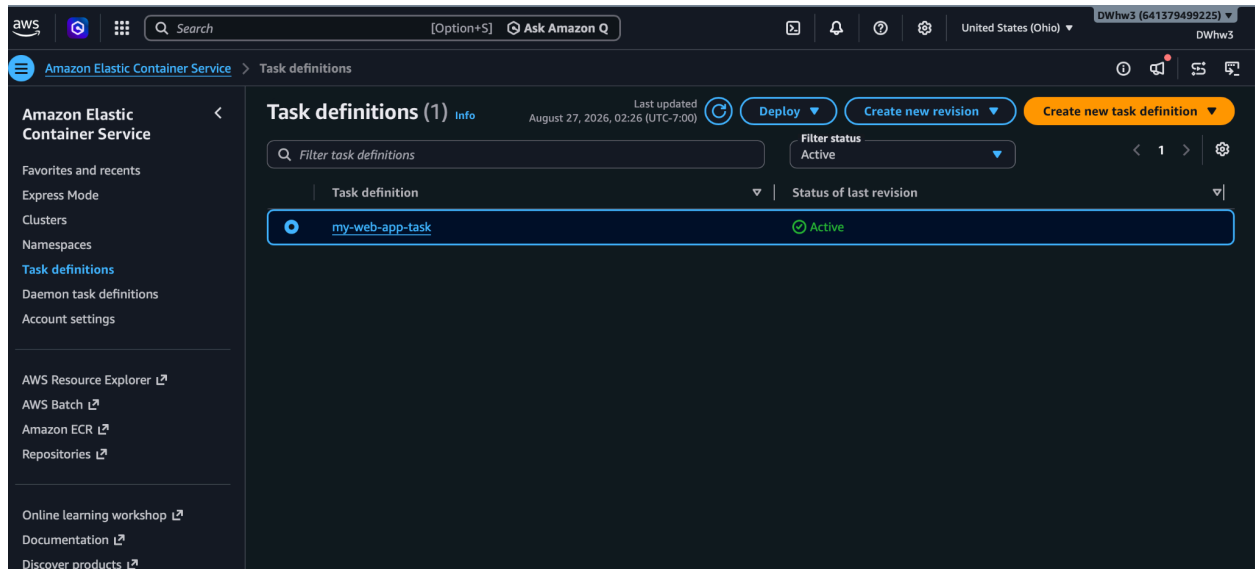
Created my-web-app-cluster (AWS Fargate, serverless). First creation attempt failed with "Unable to assume the service linked role" a known first-time-use race condition where AWS creates the AWSServiceRoleForECS role asynchronously, retrying the cluster creation succeeded once the role existed.



Task definition

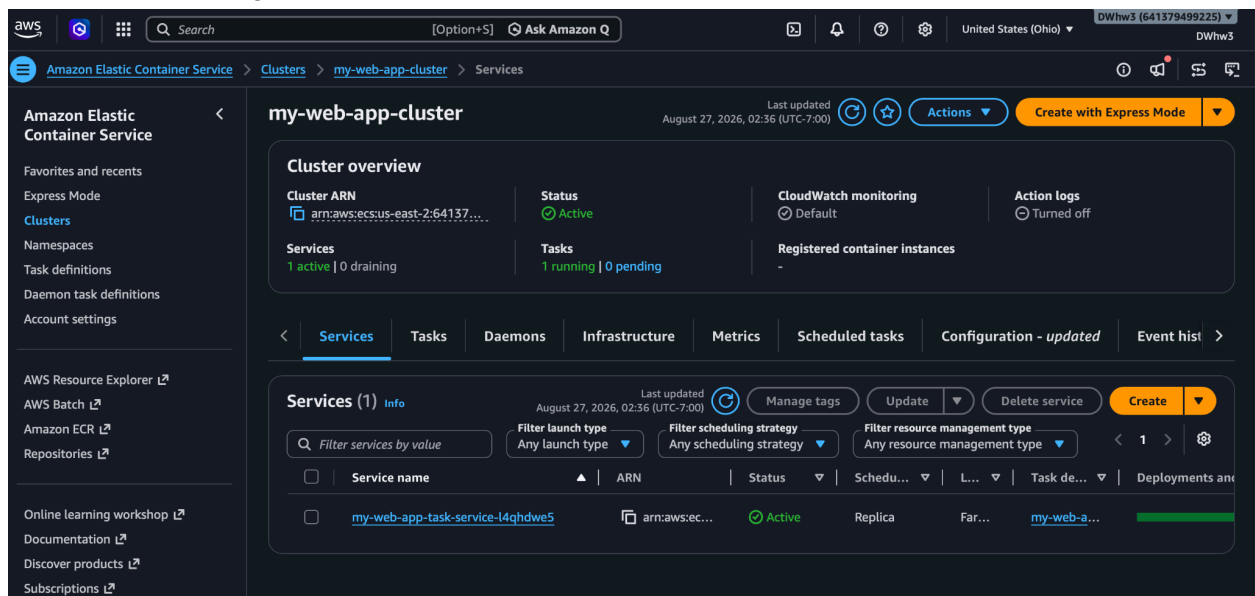
Created my-web-app-task: Fargate, Linux/ARM64 (matching the image's actual architecture

,0.25 vCPU / 0.5 GB, default ecsTaskExecutionRole auto-created, container web-app-container with image URI 641379499225.dkr.ecr.us-east-2.amazonaws.com/my-web-app:latest, container port 8260/TCP.



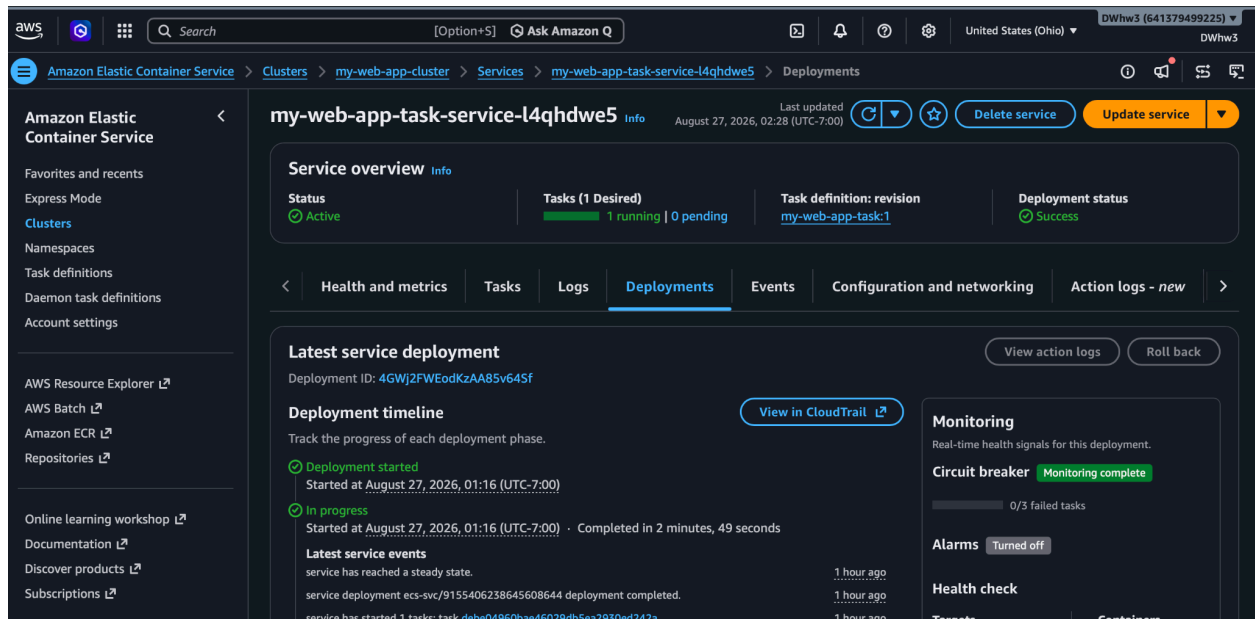
ECS Cluster

Created a my-web-app-cluster with selecting infrastructure AWS Fargate (serverless) and rest left at default setting.



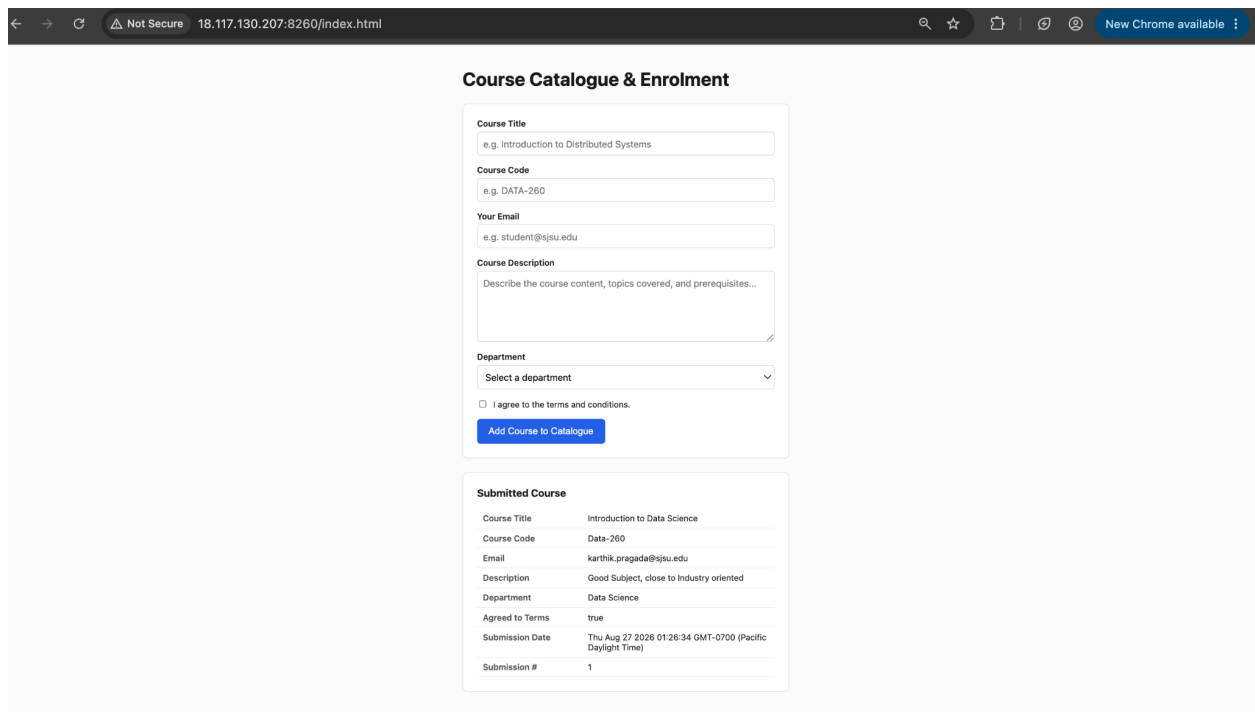
ECS service

Created my-web-app-task-service-l4qhdwe5 created with Desired tasks = 1, default VPC, all 3 default public subnets, the my-web-app-sg security group, and Public IP enabled.



Final Deployment Picture

Task reached RUNNING status with public IP 18.117.130.207. Verified reachable both via curl (HTTP 200) and directly in a browser.



Part 2 (Agentic AI)

Two tiny agents (Planner and Reviewer) that talk to each other through a local LLM served by Ollama, with a deterministic Finalizer step, producing exactly 3 topical tags and a less than 25 word summary from an arbitrary title/content pair, printed as strict JSON.

Created a virtual environment with Python 3.12

```
/opt/homebrew/bin/python3.12 -m venv .venv
source .venv/bin/activate
pip install langchain-ollama langchain-core
```

This installed langchain-core 1.6.1, langchain-ollama 1.1.0, and the underlying ollama 0.6.2 Python client. Separately, the local LLM itself was pulled through the Ollama CLI (not pip Ollama, it runs as its own background service that serves models over a local HTTP API, which langchain-ollama then talks to):

Installed qwen3:8b Ollama Model.

```
Last login: Thu Aug 27 00:36:05 on ttys007
[(base) prags@Karthiks-MacBook-Air ~ % cd /Users/prags/Desktop/HW1_DATA260
[(base) prags@Karthiks-MacBook-Air HW1_DATA260 % source .venv/bin/activate
[(.venv) ] (base) prags@Karthiks-MacBook-Air HW1_DATA260 % ollama list
NAME          ID          SIZE    MODIFIED
qwen3:8b      500a1f067a9f 5.2 GB   3 hours ago
codellama:7b  8fdf8f752f6e 3.8 GB   6 months ago
```

Architecture of agents

PLANNER_SYSTEM / REVIEWER_SYSTEM — two system-prompt strings, each fully generic. One Agent plans and the other Agent reviews the Planner Agent's generated response and suggests changes to it.

extract_json() - a small parsing helper that first tries json.loads() directly, and falls back to regex-extracting the first {...} block if the model wraps its JSON in any stray text, so the pipeline doesn't crash on minor formatting slips.

call_agent() — sends a (system, human) message pair to ChatOllama, times the call, and returns the parsed JSON plus latency in milliseconds.

run_pipeline() — orchestrates the full Planner → Reviewer → Finalizer sequence and prints each stage.

main() — CLI entry point. Accepts --title/--content

```
... # style.css <> index.html JS script.js agents_demo.py X
agents_demo.py
1  #!/usr/bin/env python3
2  """Planner -> Reviewer -> Finalizer agent pipeline over a local Ollama LLM."""
3
4  import argparse
5  import json
6  import re
7  import time
8
9  from langchain_ollama import ChatOllama
10
11  PLANNER_SYSTEM = """You are the Planner agent in a two-agent tagging pipeline.
12  Given a title and content, propose exactly 3 short topical tags (2-4 words each,
13  lowercase, specific to the content -- not generic categories) and a one-sentence
14  summary of at most 25 words that captures the key point of the content.
15  Respond with ONLY valid JSON, no other text, in this exact schema:
16  {"tags": ["tag1", "tag2", "tag3"], "summary": "..."}"""
17
18  REVIEWER_SYSTEM = """You are the Reviewer agent in a two-agent tagging pipeline.
19  You will receive the original title/content and a Planner's draft JSON containing
20  3 tags and a summary. Critically check whether the tags are specific and accurate
21  to the content (not vague or generic), and whether the summary is at most 25 words
22  and faithful to the content. If the draft already meets these standards, return it
23  unchanged. If not, return an improved version that does.
24  Respond with ONLY valid JSON, no other text, in this exact schema:
25  {"tags": ["tag1", "tag2", "tag3"], "summary": "..."}"""
26
27  DEFAULT_TITLE = "New Course: Distributed Systems Fundamentals"
28  DEFAULT_CONTENT = (
29      "This course covers the design and implementation of distributed systems, "
30      "including consensus algorithms, replication strategies, partitioning, and "
31      "fault tolerance. Students will build a small key-value store that tolerates "
32      "node failures and implement a leader election protocol. Prerequisites: "
33      "data structures and computer networks."
34  )
35
36
37  def extract_json(text: str) -> dict:
38      text = text.strip()
39      try:
40          return json.loads(text)
41      except json.JSONDecodeError:
42          pass
43      match = re.search(r"\{.*\}", text, re.DOTALL)
44      if match:
45          return json.loads(match.group(0))
46      raise ValueError(f"Could not parse JSON from model output:\n{text}")
47
48
49  def call_agent(llm, system_prompt, user_prompt):
50      messages = [("system", system_prompt), ("human", user_prompt)]
```

```

56
57
58 def run_pipeline(title, content, model, temperature, verbose=True):
59     llm = ChatOllama(model=model, temperature=temperature, format="json")
60     user_prompt = f"Title: {title}\n\nContent: {content}"
61
62     if verbose:
63         print("=" * 70)
64         print("PLANNER")
65         print("=" * 70)
66     planner_output, planner_ms, _ = call_agent(llm, PLANNER_SYSTEM, user_prompt)
67     if verbose:
68         print(json.dumps(planner_output, indent=2))
69         print(f"(latency: {planner_ms:.0f} ms)\n")
70
71     reviewer_prompt = f"{user_prompt}\n\nPlanner draft:\n{json.dumps(planner_output)}"
72
73     if verbose:
74         print("=" * 70)
75         print("REVIEWER")
76         print("=" * 70)
77     reviewer_output, reviewer_ms, _ = call_agent(llm, REVIEWER_SYSTEM, reviewer_prompt)
78     if verbose:
79         print(json.dumps(reviewer_output, indent=2))
80         print(f"(latency: {reviewer_ms:.0f} ms)\n")
81
82     reviewer_changed = planner_output != reviewer_output
83
84     # Finalization step -- deterministic, non-LLM: enforce the output contract.
85     tags = list(reviewer_output.get("tags", []))[:3]
86     while len(tags) < 3:
87         tags.append("general")
88     summary = str(reviewer_output.get("summary", "")).strip()
89     words = summary.split()
90     if len(words) > 25:
91         summary = " ".join(words[:25])
92
93     final_output = {"tags": tags, "summary": summary}
94
95     if verbose:
96         print("=" * 70)
97         print("FINALIZED (Publish)")
98         print("=" * 70)
99         print(json.dumps(final_output, indent=2))
100
101     return {

```

```

114
115 def main():
116     parser = argparse.ArgumentParser(
117         description="Planner -> Reviewer -> Finalizer agent demo over a local Ollama LLM."
118     )
119     parser.add_argument("--title", type=str, default=None)
120     parser.add_argument("--content", type=str, default=None)
121     parser.add_argument("--input", type=str, default=None, help="Path to JSON file with {title, content}")
122     parser.add_argument("--model", type=str, default="qwen3:8b")
123     parser.add_argument("--temperature", type=float, default=0.7)
124     args = parser.parse_args()
125
126     if args.input:
127         with open(args.input) as f:
128             data = json.load(f)
129             title, content = data["title"], data["content"]
130     elif args.title and args.content:
131         title, content = args.title, args.content
132     else:
133         print("No --title/--content/--input given -- enter them now.")
134         print(f"(press Enter on Title to use the default example: {DEFAULT_TITLE!r})\n")
135         title = input("Title: ").strip()
136         if not title:
137             title, content = DEFAULT_TITLE, DEFAULT_CONTENT
138         else:
139             content = input("Content: ").strip()
140         print()
141
142     run_pipeline(title, content, args.model, args.temperature)
143
144
145 if __name__ == "__main__":
146     main()
147

```

How each agent works / Explanation of each step

The Planner receives only the raw title and content, wrapped in a generic "Title: ...\n\nContent: ..." user message, and is instructed to extract exactly 3 topical tags and a less than 25-word summary. The call passes format="json" to ChatOllama, which instructs Ollama to constrain sampling to valid JSON tokens; this is what makes the "strict JSON output" requirement reliable rather than hoping the model behaves.

The Reviewer receives the same original title/content plus the Planner's draft JSON appended to the prompt, and is instructed to critically re-examine it: are the tags specific rather than generic, is the summary within the word limit and faithful to the content. It returns either the same JSON unchanged (if it judges the draft already meets the bar) or a corrected version.

Comparing planner_output != reviewer_output after this call is what answers "did the Reviewer change anything." The Finalizer is deliberately not a third LLM call it's a few lines of plain Python that mechanically enforce the output contract regardless of what the model produced: slicing the tags list to exactly 3 entries (padding with a placeholder if the model ever returned fewer), and truncating the summary to 25 words if it ever overshoots. This guarantees the published JSON is always structurally valid, even on an off run.

Result:

When we run the `python agents_demo.py`,

It asks for title and Content which the user has to dynamically input. Once title and content is passed, the planner Agent gives the tags and a Short Summary. The reviewer reviews and suggests any changes if required and the finalizer Agent finalizes the Tags and Summary.

```
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % python agents_demo.py
No --title/--content/--input given -- enter them now.
(press Enter on Title to use the default example: 'New Course: Distributed Systems Fundamentals')

Title: Introduction to Machine Learning
Content: a branch of Artificial Intelligence that allows computers to learn from data and make predictions without being explicitly programmed for every task.
=====
PLANNER
=====
{
  "tags": [
    "machine learning basics",
    "artificial intelligence branch",
    "data prediction methods"
  ],
  "summary": "Machine learning enables computers to learn from data and make predictions without explicit programming."
}
(latency: 47423 ms)
=====
REVIEWER
=====
{
  "tags": [
    "machine learning basics",
    "artificial intelligence branch",
    "data prediction methods"
  ],
  "summary": "Machine learning enables computers to learn from data and make predictions without explicit programming."
}
(latency: 24379 ms)
=====
FINALIZED (Publish)
=====
{
  "tags": [
    "machine learning basics",
    "artificial intelligence branch",
    "data prediction methods"
  ],
  "summary": "Machine learning enables computers to learn from data and make predictions without explicit programming."
}
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 %
```

The 3 final tags:

"machine learning basics", "artificial intelligence branch", "data prediction methods"

The final summary (14 words): "Machine learning enables computers to learn from data and make predictions without explicit programming."

Did the Reviewer agent change anything? No. The Reviewer returned the exact same JSON (tags and summary) that the Planner produced its own instructions saying "if the draft already meets these standards, return it unchanged," and it judged the Planner's first-pass output as already meeting the specificity and length requirements.

Part 3 - Measuring Non-Determinism

The same fixed input was run through the full `agents_demo.py` pipeline 40 times — 20 at temperature 0.7, 20 at temperature 0.0 to measure, with real numbers rather than assumption, how much a local LLM's output actually varies run-to-run at each setting.

Fixed Input

```
{  
  "title": "Data Structures and Algorithms"  
,  
  "content": "This course covers fundamental data structures including arrays, linked lists,  
stacks, queues, trees, and graphs, along with algorithm design techniques such as sorting,  
searching, divide-and-conquer, dynamic programming, and greedy methods. Students will  
Analyze time and space complexity using Big-O notation and implement each structure from  
scratch in Python. Prerequisites: introductory programming and discrete mathematics."  
}
```

Driver script

`run_nondeterminism.py` reuses `run_pipeline()` from `agents_demo.py` directly (no duplicated agent logic), looping it 20 times per temperature, catching and logging any per-run failures without aborting the sweep, then computing distinct tag sets, tag-frequency counts, and latency percentiles across the successful runs. All 40 raw records (tags, summary, per-agent latency, Reviewer-changed flag) are written to `reports/hw01/raw/nondeterminism_runs.json` and `.csv`; the aggregated statistics to `reports/hw01/raw/nondeterminism_summary.json`.

20 Runs when the temperature is at 0.7

```
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % ollama list
NAME      ID              SIZE    MODIFIED
qwen3:8b  500a1f067a9f  5.2 GB  2 days ago
codellama:7b  8fd8f752f6e  3.8 GB  6 months ago
(.venv) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % python run_nondeterminism.py

#####
# TEMPERATURE 0.7 -- 20 runs
#####

--- run 1/20 @ temp=0.7 (overall 1/40) ---
{
  "run_index": 1,
  "temperature": 0.7,
  "tags": [
    "data structure implementation",
    "algorithm design techniques",
    "python implementation"
  ],
  "summary": "Course covers data structures, algorithm techniques, and Python implementation with Big-O analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 35538.1,
  "reviewer_latency_ms": 34275.1,
  "total_latency_ms": 69813.3,
  "error": null
}

--- run 2/20 @ temp=0.7 (overall 2/40) ---
{
  "run_index": 2,
  "temperature": 0.7,
  "tags": [
    "data structures fundamentals",
    "algorithm design techniques",
    "python implementation"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on implementation in Python and complexity analysis using Big-O notation.",
  "reviewer_changed": false,
  "planner_latency_ms": 19245.6,
  "reviewer_latency_ms": 30375.3,
  "total_latency_ms": 49620.9,
  "error": null
}

--- run 3/20 @ temp=0.7 (overall 3/40) ---
{
  "run_index": 3,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o complexity analysis"
  ],
  "summary": "Course covers data structures and algorithms, their implementation in Python, and complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 57082.7,
  "reviewer_latency_ms": 24479.4,
  "total_latency_ms": 81562.1,
  "error": null
}
```

```

}
--- run 4/20 @ temp=0.7 (overall 4/40) ---
{
  "run_index": 4,
  "temperature": 0.7,
  "tags": [
    "data structure implementation",
    "algorithm design techniques",
    "time and space complexity"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on Python implementation and Big-O complexity analysis.",
  "reviewer_changed": true,
  "planner_latency_ms": 68194.6,
  "reviewer_latency_ms": 45188.8,
  "total_latency_ms": 113383.4,
  "error": null
}
--- run 5/20 @ temp=0.7 (overall 5/40) ---
{
  "run_index": 5,
  "temperature": 0.7,
  "tags": [
    "data structure implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "This course covers data structures and algorithms, including implementation in Python and analysis of time complexity.",
  "reviewer_changed": false,
  "planner_latency_ms": 28371.9,
  "reviewer_latency_ms": 25881.1,
  "total_latency_ms": 46253.0,
  "error": null
}
--- run 6/20 @ temp=0.7 (overall 6/40) ---
{
  "run_index": 6,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o complexity analysis"
  ],
  "summary": "This course covers data structures, algorithm design, and complexity analysis with Python implementation.",
  "reviewer_changed": false,
  "planner_latency_ms": 66368.9,
  "reviewer_latency_ms": 20459.5,
  "total_latency_ms": 76808.4,
  "error": null
}
--- run 7/20 @ temp=0.7 (overall 7/40) ---
{
  "run_index": 7,
  "temperature": 0.7,
  "tags": [
    "data structure implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on implementation in Python and complexity analysis using Big-O notation.",
}

"run_index": 8,
"temperature": 0.7,
"tags": [
  "linked list implementation",
  "dynamic programming methods",
  "big-o complexity analysis"
],
"summary": "This course teaches data structures and algorithms, focusing on Python implementation and Big-O complexity analysis.",
"reviewer_changed": true,
"planner_latency_ms": 40718.0,
"reviewer_latency_ms": 36351.9,
"total_latency_ms": 77070.0,
"error": null
}
--- run 9/20 @ temp=0.7 (overall 9/40) ---
{
  "run_index": 9,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on implementation in Python and complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 61312.4,
  "reviewer_latency_ms": 38277.3,
  "total_latency_ms": 99589.6,
  "error": null
}
--- run 10/20 @ temp=0.7 (overall 10/40) ---
{
  "run_index": 10,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on Python implementation and Big-O complexity analysis.",
  "reviewer_changed": true,
  "planner_latency_ms": 24894.8,
  "reviewer_latency_ms": 35920.5,
  "total_latency_ms": 60815.3,
  "error": null
}
--- run 11/20 @ temp=0.7 (overall 11/40) ---
{
  "run_index": 11,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "Course covers data structures, algorithm design techniques, and Big-O complexity analysis with Python implementation.",
  "reviewer_changed": false,

```

```

    "error": null
  }
}

--- run 12/20 @ temp=0.7 (overall 12/40) ---
{
  "run_index": 12,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "Course covers data structures and algorithms, including Python implementation, time and space complexity analysis using Big-O notation.",
  "reviewer_changed": true,
  "planner_latency_ms": 57881.7,
  "reviewer_latency_ms": 95773.9,
  "total_latency_ms": 93655.6,
  "error": null
}

--- run 13/20 @ temp=0.7 (overall 13/40) ---
{
  "run_index": 13,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm analysis",
    "python programming"
  ],
  "summary": "This course covers data structures and algorithms, focusing on Python implementation and complexity analysis.",
  "reviewer_changed": true,
  "planner_latency_ms": 45204.6,
  "reviewer_latency_ms": 38904.3,
  "total_latency_ms": 84108.9,
  "error": null
}

--- run 14/20 @ temp=0.7 (overall 14/40) ---
{
  "run_index": 14,
  "temperature": 0.7,
  "tags": [
    "data structures fundamentals",
    "algorithm design techniques",
    "python implementation"
  ],
  "summary": "This course teaches data structures and algorithms, including implementation in Python and complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 48719.4,
  "reviewer_latency_ms": 40809.2,
  "total_latency_ms": 89528.6,
  "error": null
}

--- run 15/20 @ temp=0.7 (overall 15/40) ---
{
  "run_index": 15,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],

```

```

    "data structures implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on implementation and complexity analysis using Python.",
  "reviewer_changed": false,
  "planner_latency_ms": 26868.3,
  "reviewer_latency_ms": 27477.3,
  "total_latency_ms": 54345.6,
  "error": null
}

--- run 16/20 @ temp=0.7 (overall 16/40) ---
{
  "run_index": 16,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "time complexity analysis"
  ],
  "summary": "Course covers data structures and algorithms with Python implementation and complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 53763.7,
  "reviewer_latency_ms": 21655.7,
  "total_latency_ms": 75419.4,
  "error": null
}

--- run 17/20 @ temp=0.7 (overall 17/40) ---
^[[
{
  "run_index": 17,
  "temperature": 0.7,
  "tags": [
    "data structure implementation",
    "algorithm design techniques",
    "big o analysis"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on implementation in Python and complexity analysis using Big-O notation.",
  "reviewer_changed": false,
  "planner_latency_ms": 47734.7,
  "reviewer_latency_ms": 26392.3,
  "total_latency_ms": 74127.0,
  "error": null
}

--- run 18/20 @ temp=0.7 (overall 18/40) ---
{
  "run_index": 18,
  "temperature": 0.7,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o complexity analysis"
  ],
  "summary": "course covers data structures and algorithms, implementation in Python, and complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 72632.8,
  "reviewer_latency_ms": 20889.9,
  "total_latency_ms": 92722.8,
  "error": null
}

```

```

--- run 19/20 @ temp=0.7 (overall 19/40) ---
{
  "run_index": 19,
  "temperature": 0.7,
  "tags": [
    "data structure implementation",
    "algorithm design techniques",
    "complexity analysis"
  ],
  "summary": "Course covers data structures and algorithms, their Python implementation, and time/space complexity analysis using Big-O notation.",
  "reviewer_changed": false,
  "planner_latency_ms": 55604.6,
  "reviewer_latency_ms": 21870.2,
  "total_latency_ms": 77474.8,
  "error": null
}

--- run 20/20 @ temp=0.7 (overall 20/40) ---
{
  "run_index": 20,
  "temperature": 0.7,
  "tags": [
    "data structures fundamentals",
    "algorithm design techniques",
    "python implementation"
  ],
  "summary": "This course teaches data structures and algorithms, focusing on Python implementation and complexity analysis.",
  "reviewer_changed": true,
  "planner_latency_ms": 46643.9,
  "reviewer_latency_ms": 55603.7,
  "total_latency_ms": 102247.7,
  "error": null
}

```

20 Runs when the Temperature is 0.0

```

#####
# TEMPERATURE 0.0 --- 20 runs
#####

--- run 1/20 @ temp=0.0 (overall 21/40) ---
{
  "run_index": 1,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 88860.6,
  "reviewer_latency_ms": 33331.0,
  "total_latency_ms": 122191.6,
  "error": null
}

--- run 2/20 @ temp=0.0 (overall 22/40) ---
{
  "run_index": 2,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 87662.0,
  "reviewer_latency_ms": 29756.3,
  "total_latency_ms": 117418.3,
  "error": null
}

--- run 3/20 @ temp=0.0 (overall 23/40) ---
{
  "run_index": 3,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 84983.8,
  "reviewer_latency_ms": 30471.6,
  "total_latency_ms": 115455.4,
  "error": null
}

--- run 4/20 @ temp=0.0 (overall 24/40) ---
{
  "run_index": 4,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",

```

```

    },
    "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
    "reviewer_changed": false,
    "planner_latency_ms": 86319.5,
    "reviewer_latency_ms": 28784.0,
    "total_latency_ms": 115023.5,
    "error": null
  }
}

--- run 5/20 @ temp=0.0 (overall 25/40) ---
{
  "run_index": 5,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 92328.2,
  "reviewer_latency_ms": 31247.3,
  "total_latency_ms": 123575.4,
  "error": null
}

--- run 6/20 @ temp=0.0 (overall 26/40) ---
{
  "run_index": 6,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 88284.6,
  "reviewer_latency_ms": 30632.1,
  "total_latency_ms": 118916.7,
  "error": null
}

--- run 7/20 @ temp=0.0 (overall 27/40) ---
{
  "run_index": 7,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 87746.9,
  "reviewer_latency_ms": 30353.7,
  "total_latency_ms": 118100.7,
  "error": null
}

--- run 8/20 @ temp=0.0 (overall 28/40) ---

```

```

--- run 8/20 @ temp=0.0 (overall 28/40) ---
{
  "run_index": 8,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 88640.8,
  "reviewer_latency_ms": 30341.0,
  "total_latency_ms": 118981.8,
  "error": null
}

--- run 9/20 @ temp=0.0 (overall 29/40) ---
{
  "run_index": 9,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 94788.6,
  "reviewer_latency_ms": 30403.2,
  "total_latency_ms": 131191.8,
  "error": null
}

--- run 10/20 @ temp=0.0 (overall 30/40) ---
{
  "run_index": 10,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 96838.2,
  "reviewer_latency_ms": 32226.5,
  "total_latency_ms": 128264.8,
  "error": null
}

--- run 11/20 @ temp=0.0 (overall 31/40) ---
{
  "run_index": 11,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,

```

```

    },
    "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
    "reviewer_changed": false,
    "planner_latency_ms": 88027.5,
    "reviewer_latency_ms": 30267.8,
    "total_latency_ms": 118295.2,
    "error": null
  }
}

--- run 12/20 @ temp=0.0 (overall 32/40) ---
{
  "run_index": 12,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 84625.1,
  "reviewer_latency_ms": 29414.9,
  "total_latency_ms": 113940.1,
  "error": null
}

--- run 13/20 @ temp=0.0 (overall 33/40) ---
{
  "run_index": 13,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 85106.0,
  "reviewer_latency_ms": 29486.4,
  "total_latency_ms": 114592.4,
  "error": null
}

--- run 14/20 @ temp=0.0 (overall 34/40) ---
{
  "run_index": 14,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 86272.4,
  "reviewer_latency_ms": 29545.0,
  "total_latency_ms": 115817.5,
  "error": null
}

--- run 15/20 @ temp=0.0 (overall 35/40) ---
{
  "run_index": 15,

```

```

--- run 15/20 @ temp=0.0 (overall 35/40) ---
{
  "run_index": 15,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 87110.3,
  "reviewer_latency_ms": 31025.5,
  "total_latency_ms": 118135.8,
  "error": null
}

--- run 16/20 @ temp=0.0 (overall 36/40) ---
{
  "run_index": 16,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 101974.4,
  "reviewer_latency_ms": 33869.0,
  "total_latency_ms": 135843.4,
  "error": null
}

--- run 17/20 @ temp=0.0 (overall 37/40) ---
{
  "run_index": 17,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 93012.1,
  "reviewer_latency_ms": 30075.0,
  "total_latency_ms": 123087.0,
  "error": null
}

--- run 18/20 @ temp=0.0 (overall 38/40) ---
{
  "run_index": 18,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,

```

```

--- run 19/20 @ temp=0.0 (overall 39/40) ---
{
  "run_index": 19,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 99708.8,
  "reviewer_latency_ms": 38888.1,
  "total_latency_ms": 138597.0,
  "error": null
}

--- run 20/20 @ temp=0.0 (overall 40/40) ---
{
  "run_index": 20,
  "temperature": 0.0,
  "tags": [
    "data structures implementation",
    "algorithm design techniques",
    "big o notation"
  ],
  "summary": "This course covers data structures and algorithms, including Python implementation and Big-O complexity analysis.",
  "reviewer_changed": false,
  "planner_latency_ms": 97590.6,
  "reviewer_latency_ms": 35384.6,
  "total_latency_ms": 132975.3,
  "error": null
}

```

Summary

```

=====
SUMMARY
=====

--- Temperature 0.7 (20 successful / 0 failed) ---
{
  "successful_runs": 20,
  "failed_runs": 0,
  "distinct_tag_sets": 11,
  "tags_in_all_runs": [],
  "tags_in_exactly_one_run": [
    "algorithm analysis",
    "big o analysis",
    "big o notation",
    "big-o complexity analysis",
    "complexity analysis",
    "dynamic programming methods",
    "linked list implementation",
    "python programming",
    "time and space complexity"
  ],
  "latency_p50_ms": 77272.4,
  "latency_p95_ms": 102800.5,
  "latency_p99_ms": 111202.8
}

--- Temperature 0.0 (20 successful / 0 failed) ---
{
  "successful_runs": 20,
  "failed_runs": 0,
  "distinct_tag_sets": 1,
  "tags_in_all_runs": [
    "algorithm design techniques",
    "big o notation",
    "data structures implementation"
  ],
  "tags_in_exactly_one_run": [],
  "latency_p50_ms": 118949.2,
  "latency_p95_ms": 135981.1,
  "latency_p99_ms": 138073.8
}

Saved summary to reports/hw01/raw/nondeterminism_summary.json
((.env) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % █

```

What two users sending identical input might see:

At **temperature 0.0**, two users would see the *exact same* output every time across all 20 runs, the pipeline converged on one single tag set: "algorithm design techniques", "big o notation", "data structures implementation", with zero variation observed. Temperature 0.0 essentially forces greedy decoding, so identical input reliably produces identical output.

At **temperature 0.7**, two users would very likely see genuinely different results. Out of 20 runs on the same input, 11 distinct tag sets came out more than half the runs produced a unique combination and not a single tag was common to all 20 runs. So one user might get "linked list implementation" and "python programming" as tags, while another user, asking the exact same question, gets "big-o complexity analysis" and "dynamic programming methods" instead. The only thing they could count on is a loose family resemblance (most tags orbit the same handful of concepts), not identical wording.

Example where this variation is acceptable: a "suggested tags" feature in a course-authoring tool, where the output is shown to a human instructor as a *draft* they review and edit before publishing. Here, the variety at temperature 0.7 is actually useful, different phrasings each run give the instructor more options to pick from, and because a human checks it before it goes live, inconsistency between runs never reaches an end user.

Example where this variation is not acceptable: if these tags were used automatically to route the course into search-index categories with no human review. A student searching the catalog for "linked list implementation" would find this course in some runs but not others, even though the underlying course description never changed a broken, confusing experience caused entirely by sampling randomness. This is exactly the kind of case where temperature=0.0 should be pinned instead, since anything feeding an automated decision without a human checkpoint needs to be reproducible.

Metric	Temp 0.7	Temp 0.0
Distinct tag sets	11	1
Tags in all 20 runs	none	"algorithm design techniques", "big o notation", "data structures implementation"
Tags in exactly 1 run	"algorithm analysis", "big o analysis", "big o notation", "big-o complexity analysis", "complexity analysis", "dynamic programming methods", "linked list implementation", "python programming", "time and space complexity" (9 tags)	"algorithm design techniques", "big o notation", "data structures implementation"
Latency p50 / p95 / p99 (ms)	77,272.4 / 102,800.5 / 111,202.8	118,949.2 / 135,981.1 / 138,073.8

Part 4 — Model Client and Token Accounting

A reusable model-adapter module, a small CLI chat demo built on it, a system-prompt file enforcing a strict response format, and real per-turn and cumulative token accounting.

src/model_client.py — the adapter

Defines ModelClient, exposing exactly one stable method, complete(messages, tools=None) that every other script in the project must call through. Nothing outside this file imports langchain_ollama directly, so swapping the underlying provider later only means changing this one module.

```
HW1_DATA260

# style.css  index.html  JS script.js  model_client.py X

src > model_client.py
1  from dataclasses import dataclass
2  from typing import Optional
3
4  from langchain_ollama import ChatOllama
5
6
7  @dataclass
8  class CompletionResult:
9      content: str
10     input_tokens: int
11     output_tokens: int
12     total_tokens: int
13
14
15  class ModelClient:
16     """Adapter over a local Ollama chat model, exposing one stable method: complete()."""
17
18     def __init__(self, model: str = "qwen3:8b", temperature: float = 0.0):
19         self.model = model
20         self.temperature = temperature
21         self._llm = ChatOllama(model=model, temperature=temperature, reasoning=False)
22
23         self.turn_count = 0
24         self.cumulative_input_tokens = 0
25         self.cumulative_output_tokens = 0
26
27     def complete(self, messages: list[tuple[str, str]], tools: Optional[list] = None) -> CompletionResult:
28         """messages: list of (role, content) tuples, e.g. [("system", "..."), ("user", "...")].
29         tools: reserved for future tool-calling support; unused by this adapter today."""
30         llm = self._llm.bind_tools(tools) if tools else self._llm
31         response = llm.invoke(messages)
32
33         usage = response.usage_metadata or {}
34         input_tokens = usage.get("input_tokens", 0)
35         output_tokens = usage.get("output_tokens", 0)
36         total_tokens = usage.get("total_tokens", input_tokens + output_tokens)
37
38         self.turn_count += 1
39         self.cumulative_input_tokens += input_tokens
40         self.cumulative_output_tokens += output_tokens
41
42         return CompletionResult(
43             content=response.content,
44             input_tokens=input_tokens,
45             output_tokens=output_tokens,
46             total_tokens=total_tokens,
47         )
```

AGENT.md — strict bullet-only review instruction

```
HW1_DATA260

# style.css  index.html  JS script.js  model_client.py  AGENT.md X

AGENT.md > abc # AGENT.md
1  # AGENT.md
2
3  You are a code reviewer. When asked to review code, respond using only a bullet-point list.
4
5  Rules:
6  - Every line of your response must start with '-'.
7  - No introductory sentence, no summary paragraph, no closing remarks.
8  - No prose outside of bullets. No headings.
9  - Each bullet should state one specific issue or observation, concisely (one line each where possible).
10 - If the code has no issues, respond with a single bullet saying so — do not pad with unnecessary bullets.
11
```

hw1_client.py — the CLI demo

Loads AGENT.md as the system prompt, keeps its own conversation history, and calls `client.complete(history)` on every turn.

```
... # style.css <> index.html JS script.js model_client.py hw1_client.py X
hw1_client.py
1  import json
2  from pathlib import Path
3
4  from src.model_client import ModelClient
5
6  AGENT_MD_PATH = Path("AGENT.md")
7
8
9  def load_system_prompt() -> str:
10     return AGENT_MD_PATH.read_text()
11
12
13  def print_turn_usage(result):
14     print(f"[tokens] input: {result.input_tokens} output: {result.output_tokens} total: {result.total_tokens}")
15
16
17  def print_stats(client, history):
18     stats = client.stats()
19     serialized_len = len(json.dumps(history))
20     print("\n----- /stats -----")
21     print(f"Turn count           : {stats['turn_count']}")
22     print(f"Cumulative input tokens   : {stats['cumulative_input_tokens']}")
23     print(f"Cumulative output tokens  : {stats['cumulative_output_tokens']}")
24     print(f"Cumulative total tokens   : {stats['cumulative_total_tokens']}")
25     print(f"Serialized history length : {serialized_len} chars")
26     print("-----\n")
27
28
29  def main():
30     client = ModelClient(model="qwen3:8b", temperature=0.0)
31     history = [("system", load_system_prompt())]
32
33     print("hw1_client -- type a message, '/stats' for usage, or 'exit'/'quit' to end.\n")
34
35     while True:
36         try:
37             user_input = input("You: ").strip()
38         except EOFError:
39             break
40
41         if not user_input:
42             continue
43         if user_input.lower() in ("exit", "quit"):
44             break
45         if user_input == "/stats":
46             print_stats(client, history)
47             continue
48
49         history.append(("user", user_input))
50         print(f"\n=== Turn {client.turn count + 1} ===")
```

Result:

Ran the hw_client.py

```
(base) prags@Karthiks-MacBook-Air ~ % cd /Users/prags/Desktop/HW1_DATA260
source .venv/bin/activate
python hw1_client.py
hw1_client -- type a message, '/stats' for usage, or 'exit'/'quit' to end.

You: Review this: total = sum([n for n in numbers if n > 0])

=== Turn 1 ===
User: Review this: total = sum([n for n in numbers if n > 0])
Assistant:
- The list comprehension creates a new list, which may be inefficient for large datasets. Consider using a generator expression instead for better memory efficiency.

[tokens] input: 146  output: 29  total: 175

You: Review this: def get_item(cart, index): return cart[index]

=== Turn 2 ===
User: Review this: def get_item(cart, index): return cart[index]
Assistant:
- The function lacks error handling for cases where the index is out of bounds, which could raise an IndexError.

[tokens] input: 198  output: 23  total: 221

You: Review this: api_key = "sk-live-4f9a2b8c" # hardcoded for testing

=== Turn 3 ===
User: Review this: api_key = "sk-live-4f9a2b8c" # hardcoded for testing
Assistant:
- Hardcoding the API key is insecure and not suitable for production environments. Consider using environment variables or a secure secrets management solution.

[tokens] input: 254  output: 27  total: 281

You: /stats

----- /stats -----
Turn count           : 3
Cumulative input tokens : 598
Cumulative output tokens : 79
Cumulative total tokens : 677
Serialized history length : 1236 chars
-----

You: Review this: result = []; for x in data: result.append(x * 2)

=== Turn 4 ===
User: Review this: result = []; for x in data: result.append(x * 2)
Assistant:
- The list is being initialized as an empty list, then appended to in a loop. A more Pythonic approach would be to use a list comprehension for conciseness and readability.

[tokens] input: 308  output: 38  total: 346

You: Summarize all the issues you found across our conversation.

=== Turn 5 ===
User: Summarize all the issues you found across our conversation.
Assistant:
- The list comprehension in the first example could be replaced with a generator for better memory efficiency.
- The `get_item` function lacks error handling for out-of-bounds indices.
- Hardcoding an API key is insecure and should be avoided in production.
- The loop and list append approach in the last example is less Pythonic compared to using a list comprehension.

[tokens] input: 367  output: 77  total: 444

You: /stats

----- /stats -----
Turn count           : 5
Cumulative input tokens : 1273
Cumulative output tokens : 194
Cumulative total tokens : 1467
Serialized history length : 1973 chars
-----

You: exit

===== FINAL TOTALS =====
Total turns           : 5
Cumulative input tokens : 1273
Cumulative output tokens : 194
Cumulative total tokens : 1467
((.venv) ) (base) prags@Karthiks-MacBook-Air HW1_DATA260 % █
```

Result Table

	After turn 3	After turn 5
turn_count	3	5
cumulative_input_tokens	598	1,273
cumulative_output_tokens	79	194
cumulative_total_tokens	677	1,467
serialized_history_length_chars	1,236	1,973

Why is prior conversation context resent with every turn?

The model is stateless between calls so it has no memory of earlier turns unless that content is physically included in the current request. Each call is an independent forward pass over exactly the tokens it's given, so the client must resend the full history (system prompt + every prior user/assistant message) on every turn for the model to "remember" anything.

How is a system prompt different from a user message?

A system prompt sets persistent, session-wide behavioral rules (format, tone, constraints) and is treated as foundational configuration rather than something said within the conversation. A user message is an actual conversational turn the model is expected to respond to. Here, AGENT.md's bullet-only rule is the system prompt governing every response, while each "Review this: ..." line is a distinct user turn.

Why do input tokens grow over a conversation?

Because every turn's input is a strict superset of the previous turn's input (the whole prior history) plus whatever new message was just added. This was directly observed: input tokens climbed 146 → 198 → 254 → 308 → 367 across the 5 turns, tracking the growing history exactly.

What eventually limits that growth?

The model's context window shows the maximum tokens it can accept in one call (4096 for this qwen3:8b configuration, per ollama ps). Once history plus a new message would exceed that limit, something must give: truncation, summarization, or the call failing outright. Cost and latency also motivate managing context well before that hard ceiling in practice.